

UNIVERSIDAD DIEGO PORTALES



FACULTAD DE INGENIERÍA Y CIENCIAS

TAREAS 2 Y 3: ANÁLISIS, INTERCEPTACIÓN Y MODIFICACIÓN DEL PROTOCOLO HTTP MEDIANTE CONTENEDORES DOCKER Y SCAPY

Estudiantes:

Nicolás Esteban González Fontecilla

David Benjamin Fuentes Castro

1. Tarea 2: Análisis del Protocolo HTTP mediante Contenedores Docker

Introducción

El desarrollo técnico de las telecomunicaciones modernas descansa fundamentalmente sobre la robustez de las arquitecturas de red y la correcta implementación de sus protocolos de comunicación. Dentro de la capa de aplicación, el Protocolo de Transferencia de Hipertexto (HTTP) representa el pilar fundamental para el intercambio de información distribuida e hipermedia en la red global. Este informe técnico aborda el análisis práctico y empírico de dicho protocolo, empleando un entorno controlado y aislado para observar el comportamiento dinámico de los flujos de datos.

El propósito central de esta actividad es examinar los patrones de comportamiento de HTTP mediante una dupla de software heterogénea bajo el modelo cliente-servidor, utilizando Nginx como demonio servidor y GNU Wget como la herramienta cliente encargada de consumir el recurso. Con el fin de mitigar interferencias externas provenientes del sistema operativo anfitrión y garantizar la reproducibilidad del experimento, la infraestructura se despliega mediante técnicas de contenedorización de procesos sobre una red virtual dedicada.

A lo largo de esta primera parte se desglosará en primer lugar el sustento teórico subyacente al protocolo HTTP y la configuración explícita de los entornos virtualizados en un sistema operativo basado en Arch Linux. Posteriormente, se presentarán las capturas analíticas obtenidas a través del analizador de paquetes Wireshark, discutiendo minuciosamente los encabezados y códigos de estado observados en las transacciones de datos. Finalmente, se expondrán diversas hipótesis y análisis deductivos relacionados con la inyección o alteración de tráfico en tiempo real y su impacto en la estabilidad de las aplicaciones involucradas, hipótesis que se ponen a prueba de forma práctica en la Tarea 3 (Sección 2).

Desarrollo

Conceptos Teóricos y Tecnologías Utilizadas

El protocolo HTTP es un protocolo de la capa de aplicación (Capa 7 del modelo de referencia OSI) caracterizado por su naturaleza sin estado (*stateless*), lo que implica que cada transacción ejecutada es tratada de forma independiente, careciendo de memoria intrínseca respecto a peticiones previas. HTTP basa su comunicación en un paradigma síncrono de petición-respuesta (*request-response*), interactuando de forma directa sobre la capa de transporte mediante el protocolo orientado a la conexión TCP, comúnmente a través del puerto bien conocido 80 para sus variantes no cifradas.

La arquitectura de contenedores, gestionada a través de Docker, permite instanciar espacios de nombres (*namespaces*) y grupos de control (*cgroups*) aislados del núcleo Linux. Al interconectar estos contenedores por medio de una red virtual de tipo puente (*bridge*), se emula con precisión el comportamiento de un segmento de red física local (LAN). Esto facilita el aislamiento estricto del tráfico generado por el demonio del servidor web y las llamadas del cliente, permitiendo que un sniffer de red examine exclusivamente las unidades de datos del protocolo (PDU) deseadas.

Para el análisis del protocolo HTTP se seleccionó la combinación de Nginx como servidor web y GNU Wget como cliente HTTP. Nginx actúa como proveedor del servicio web, respondiendo solicitudes realizadas por los clientes mediante el protocolo HTTP. Por su parte, Wget permite generar solicitudes HTTP desde línea de comandos de manera simple y controlada. La elección de esta dupla cliente-servidor facilita la observación y análisis del tráfico intercambiado, permitiendo identificar claramente las solicitudes realizadas por el cliente y las respuestas generadas por el servidor.

Tráfico Esperado

Antes de realizar las capturas se esperaba observar el establecimiento de una conexión TCP entre cliente y servidor mediante el proceso conocido como *three-way handshake*. Una vez establecida la conexión, se esperaba visualizar una solicitud HTTP utilizando el método GET enviada por el cliente Wget hacia el servidor Nginx.

Posteriormente, el servidor debía responder con un mensaje HTTP 200 OK acompañado por las cabeceras correspondientes y el contenido de la página web por defecto de Nginx. Finalmente, se esperaba observar el cierre de la conexión TCP mediante los segmentos de terminación correspondientes.

Debido a que ambos contenedores se encuentran conectados a una red virtual tipo *bridge* administrada por Docker, todo el tráfico generado debía permanecer dentro de dicha red virtual, facilitando su captura y análisis

mediante Wireshark.

Procedimientos Realizados y Configuración de Software

El entorno experimental se construyó sobre un sistema anfitrión con distribución Arch Linux. Para el levantamiento del entorno y garantizar el control total sobre las dependencias de los contenedores, se optó por la creación de imágenes propias mediante archivos **Dockerfile**, utilizando Ubuntu como sistema operativo base.

Servidor El servicio servidor se construyó sobre el demonio Nginx. A continuación, el **Dockerfile** utilizado para configurar la imagen del servidor:

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y nginx
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Cliente El servicio cliente se configuró instalando la herramienta GNU Wget. El contenedor se mantiene en ejecución en segundo plano para permitir la invocación interactiva de comandos desde la terminal del anfitrión:

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y wget
CMD ["tail", "-f", "/dev/null"]
```

Conexión Cliente-Servidor Para establecer la infraestructura de red aislada y ejecutar las imágenes construidas, se ejecutaron los siguientes comandos secuenciales en la terminal. Estos permiten crear la red virtual, construir las imágenes y levantar los contenedores:

```
# 1. Creacion de una red virtual en Docker
docker network create red_tarea2

# 2. Construccion de imagenes
docker build -t ubuntu-nginx ./servidor
docker build -t ubuntu-wget ./cliente

# 3. Levantamiento de contenedores conectados a la red
docker run -d --name servidor_http --network red_tarea2 ubuntu-nginx
docker run -d --name cliente_http --network red_tarea2 ubuntu-wget

# 4. Verificacion de contenedores activos
docker ps
```

Como se observa en los anexos, los contenedores fueron desplegados correctamente y permanecieron activos durante toda la ejecución del experimento, permitiendo establecer la comunicación cliente-servidor necesaria para el análisis del protocolo HTTP.

Posteriormente, se procedió a identificar la interfaz de red virtual autogenerada por el controlador *bridge* de Docker mediante la consulta de enlaces de red del *kernel* (*ip link*). Localizado el identificador exacto de la interfaz (**br-2ed43d614f56**), se ejecutó el analizador Wireshark aplicando de forma estricta un filtro de visualización **http** para aislar los paquetes de la capa de aplicación. Con la captura activa, se inyectó tráfico mediante la invocación remota de una petición HTTP desde el contenedor cliente hacia el servidor web Nginx empleando la herramienta Wget, redirigiendo la salida estándar al dispositivo nulo:

```
# Inyeccion de solicitud de trafico desde el cliente hacia el servidor Nginx
docker exec cliente_http wget -O /dev/null http://servidor_http
```

Análisis de Tráfico Observado

La Figura 5 muestra la captura obtenida mediante Wireshark sobre la red virtual creada por Docker. En ella puede observarse la solicitud HTTP enviada por el cliente Wget y la respuesta generada por el servidor Nginx.

La captura de datos crudos arrojada por Wireshark (cuyas evidencias gráficas se adjuntan formalmente en la sección de Anexos, en conformidad con las pautas estructurales del informe) refleja con total nitidez el ciclo de comunicación HTTP/1.1 elemental. Al aplicar el filtro de capa de aplicación, el analizador discrimina el tráfico subyacente de control de flujo de la capa de transporte (tales como los segmentos de sincronización y reconocimiento del *3-way handshake* de TCP) y expone únicamente las transacciones HTTP.

El análisis de la PDU capturada revela dos hitos operacionales críticos:

1. **Petición del Cliente (Paquete 4):** Proveniente de la dirección IP de origen **172.19.0.3** (asignada dinámicamente al contenedor **cliente_http**) con destino a la dirección IP **172.19.0.2** (**servidor_http**). Se observa el método **GET / HTTP/1.1**, acompañado por campos de cabecera característicos como **User-Agent: Wget/[versión]**, el campo obligatorio **Host: servidor_http** que permite la resolución de hosts virtuales en el servidor, y la directiva **Connection: Keep-Alive** definida por la lógica interna de Wget.
2. **Respuesta del Servidor (Paquete 6):** Emitida por la IP del servidor hacia el cliente. El mensaje de control inicia con la línea de estado **HTTP/1.1 200 OK**, ratificando que la solicitud fue procesada con éxito y que el recurso solicitado está disponible. En el desglose de cabeceras se identifica el campo **Server: nginx/[versión]**, metadatos de sincronización temporal (**Date**), el tipo MIME del objeto transferido (**Content-Type: text/html**) y la longitud exacta del cuerpo del mensaje en bytes (**Content-Length**). Inmediatamente después de la doble línea de retorno de carro y salto de línea (**\r\n**), se adjunta la carga útil (*payload*) correspondiente al código de la página web de bienvenida por defecto de Nginx.

Modificación de Tráfico y Análisis de Métricas (Suposiciones del Escenario)

En el contexto del análisis avanzado de redes, la interceptación y manipulación de paquetes sobre el canal de comunicación (*on-the-fly*) representa una técnica crítica tanto para la evaluación de la resiliencia del software como para auditorías de seguridad. Si se emplearan herramientas de enrutamiento intermedio o proxies transparentes capaces de reescribir los bits de la PDU HTTP en tránsito, las implicaciones en el comportamiento de los demonios de software serían severas y asimétricas.

- **Alteración de la Línea de Estado en la Respuesta:** Si un agente intermedio interceptara el Paquete emitido por Nginx y alterara la cadena binaria de la línea de estado, sustituyendo el código **200 OK** por un código de error de la familia de cliente como **403 Forbidden** o **404 Not Found**, la repercusión en el servidor Nginx sería nula, dado que para sus registros internos la transacción concluyó con éxito. Sin embargo, en el extremo del cliente, la lógica interna de Wget procesaría el nuevo código de estado mutado, interrumpiendo abruptamente el almacenamiento del flujo de datos en el disco duro, imprimiendo un mensaje de error crítico de acceso denegado o recurso inexistente en la consola de comandos de Arch Linux y finalizando su proceso con un código de salida distinto de cero (*non-zero exit status*).
- **Manipulación de Cabeceras de Longitud de Datos (Content-Length):** Si se alterara la cabecera **Content-Length** modificando su valor nominal a uno significativamente inferior al tamaño real del archivo HTML enviado por Nginx, el software Wget leería únicamente la cantidad de bytes especificada en el header manipulado, descartando el resto del flujo de datos e interpretando que la transacción finalizó correctamente, lo que resultaría en una descarga de archivo truncada y corrupta. Por el contrario, si el valor se modificara de forma maliciosa a una escala superior al tamaño real del recurso, Wget mantendría el socket TCP abierto en estado de escucha activa, esperando recibir los bytes faltantes hasta que se active el mecanismo de temporización por inactividad (*timeout*) del software cliente, degradando las métricas de eficiencia temporal de la red.

Estas dos hipótesis, formuladas de manera teórica en esta primera etapa, se retoman y verifican de forma práctica mediante Scapy en la Tarea 3 (Sección 2).

Conclusiones

La ejecución práctica de esta actividad permitió contrastar los conceptos teóricos asociados al protocolo HTTP con su implementación real en un entorno controlado basado en contenedores Docker. Mediante la interacción entre un servidor Nginx y un cliente GNU Wget fue posible observar directamente el modelo de comunicación cliente-servidor característico de este protocolo y analizar las unidades de datos intercambiadas durante una transacción HTTP.

El uso de Wireshark permitió identificar los principales elementos involucrados en la comunicación, incluyendo la solicitud HTTP GET generada por el cliente y la respuesta HTTP 200 OK emitida por el servidor. Asimismo, se comprobó la estrecha relación existente entre HTTP y TCP, observando tanto el intercambio de mensajes de aplicación como el establecimiento y cierre de la conexión de transporte.

Una de las principales ventajas observadas de la contenedorización radica en su capacidad para aislar el tráfico generado por los servicios bajo estudio, permitiendo que la captura en Wireshark se enfoque exclusivamente en las comunicaciones relevantes para el experimento. Esto facilitó el análisis de los paquetes intercambiados y mejoró la reproducibilidad de las pruebas realizadas.

Por otra parte, el análisis de los posibles efectos derivados de la modificación del tráfico permitió comprender la importancia de los campos de control presentes en los mensajes HTTP. Alteraciones en códigos de estado o cabeceras como **Content-Length** pueden provocar comportamientos inesperados en los clientes, afectando la correcta recepción e interpretación de los recursos solicitados. Estas hipótesis, planteadas aquí de forma teórica, se ponen a prueba de manera práctica en la Tarea 3, descrita a continuación.

2. Tarea 3: Interceptación, Inyección y Modificación de Tráfico HTTP mediante Scapy

Introducción

En la Tarea 2 (Sección 2) se levantó un entorno de contenedores Docker con un servidor Nginx y un cliente GNU Wget, comunicándose mediante el protocolo HTTP sobre una red virtual tipo *bridge*. Aquello permitió observar, mediante Wireshark, el comportamiento esperado del protocolo bajo condiciones normales de operación: el establecimiento de la conexión TCP, el intercambio de la solicitud **GET** y la respuesta **200 OK**, y el cierre ordenado de la conexión.

En esta segunda etapa se busca ir un paso más allá del análisis pasivo del tráfico, interviniendo de forma activa la comunicación entre ambas aplicaciones. Para ello se utiliza Scapy, una librería de Python orientada a la manipulación de paquetes de red, que permite tanto interceptar el tráfico HTTP generado por el cliente y el servidor como inyectar tráfico malformado (*fuzzing*) o modificar en tiempo real campos específicos del protocolo. El objetivo es evaluar la resiliencia de Nginx y Wget frente a escenarios no contemplados por el diseño original del protocolo, así como identificar cotas de desempeño ante la degradación de métricas de red relevantes, en este caso, latencia y pérdida de paquetes.

A lo largo de esta parte se describe primero el entorno y la metodología utilizada para interceptar el tráfico con Scapy. Luego se detallan las dos inyecciones de tráfico mediante técnicas de *fuzzing* y las tres modificaciones realizadas sobre campos específicos del protocolo HTTP, fundamentando en cada caso el comportamiento esperado y, de no observarse, la hipótesis que explicaría la desviación. Finalmente, se presentan las dos métricas de red seleccionadas (distintas a throughput y goodput), sus cotas de desempeño y su relación con el throughput del enlace.

Desarrollo

Entorno y Configuración de Scapy

Sobre la infraestructura desplegada en la Tarea 2 (contenedores `servidor_http` y `cliente_http` conectados a la red virtual `red_tarea2`), se agregó un tercer contenedor denominado `scapy_mitm`, encargado de ejecutar todas las operaciones de interceptación, inyección y modificación de tráfico. Este contenedor se construyó a partir de una imagen Ubuntu, incorporando Python 3, la librería Scapy, la librería `NetfilterQueue` (*binding* de Python para `libnetfilter_queue`) y las utilidades `iptables` e `iproute2`:

```
FROM ubuntu:latest

RUN apt-get update && apt-get install -y \
    python3 python3-pip python3-venv \
    iptables iproute2 tcpdump net-tools \
    build-essential libnetfilter-queue-dev \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt /app/requirements.txt
RUN pip3 install --break-system-packages -r requirements.txt

COPY . /app

CMD ["tail", "-f", "/dev/null"]
```

El contenedor `scapy_mitm` se conectó a la misma red virtual `red_tarea2` utilizada por `servidor_http` y `cliente_http`, otorgándosele además las capacidades de Linux `NET_ADMIN` y `NET_RAW`, necesarias para manipular la tabla de reenvío de paquetes, instalar reglas de `iptables` y construir/enviar paquetes en bruto (*raw sockets*):

```

services:
  scapy-mitm:
    build: ./scapy-mitm
    container_name: scapy_mitm
    cap_add:
      - NET_ADMIN
      - NET_RAW
    sysctls:
      - net.ipv4.ip_forward=1
    networks:
      - red_tarea2
    command: tail -f /dev/null

```

De forma análoga, al contenedor `cliente_http` se le agregó la capacidad `NET_ADMIN` y el paquete `iproute2`, requeridos posteriormente para la manipulación de métricas de red mediante `tc netem` (Sección 2).

Metodología de Interceptación de Tráfico

A diferencia de un análisis puramente pasivo (*sniffing*), el enunciado de la Tarea 3 exige interceptar el tráfico *y* modificarlo en tránsito antes de que llegue a destino. Dado que `servidor_http` y `cliente_http` se encuentran conectados a la misma red virtual de capa 2 (*bridge* de Docker), no existe un enrutador intermedio de forma natural por el cual deba pasar el tráfico. Para forzar dicho paso se implementó la técnica de **ARP Spoofing**: el contenedor `scapy_mitm` envía periódicamente respuestas ARP falsificadas a ambos extremos, indicándole a cada uno que la dirección MAC del otro corresponde en realidad a la propia. De esta forma, todo el tráfico de capa 2 que antes viajaba directamente entre cliente y servidor pasa a ser reenviado a través de `scapy_mitm`.

```

1 def envenenar(ip_objetivo, mac_objetivo, ip_suplantada):
2     paquete = ARP(op=2, pdst=ip_objetivo, hwdst=mac_objetivo, psrc=ip_suplantada)
3     send(paquete)
4
5 def loop_spoof(cliente_ip, servidor_ip, mac_cliente, mac_servidor):
6     while not stop_event.is_set():
7         envenenar(cliente_ip, mac_cliente, servidor_ip) # cliente cree que somos el servidor
8         envenenar(servidor_ip, mac_servidor, cliente_ip) # servidor cree que somos el cliente
9         time.sleep(2)

```

Una vez posicionado como intermediario, se habilitó el reenvío de paquetes IP (`net.ipv4.ip_forward`) dentro del contenedor y se redirigió el tráfico de la cadena `FORWARD` de `iptables` hacia una cola de espacio de usuario (`NFQUEUE`), lo que permite que un proceso Python basado en Scapy y `NetfilterQueue` inspeccione cada paquete, lo modifique si corresponde al patrón del escenario en evaluación, y lo reenvíe hacia su destino real:

```

sysctl -w net.ipv4.ip_forward=1
iptables -I FORWARD -j NFQUEUE --queue-num 0
python3 mitm_modify.py --scenario status_code

```

Se verificó el correcto envenenamiento de la tabla ARP consultando la caché de vecinos del cliente (`ip neigh`), confirmando que tanto la IP del servidor como la del propio contenedor `scapy_mitm` quedaron asociadas a la misma dirección MAC (la de `scapy_mitm`), evidenciando que el tráfico efectivamente pasaría por el punto de interceptación antes de llegar a su destino real.

Inyecciones de Tráfico mediante Fuzzing

De acuerdo con lo solicitado, se realizaron dos inyecciones de tráfico utilizando técnicas de *fuzzing*, es decir, generando paquetes HTTP con contenido malformado, inesperado o fuera de especificación, con el fin de observar la reacción de Nginx ante entradas no válidas. A diferencia de las modificaciones en tránsito (Sección siguiente), estas inyecciones no requieren ARP spoofing: se construye manualmente, con Scapy, una conexión TCP completa (*three-way handshake*) y se envía el payload malformado directamente al servidor, sin pasar por Wget.

Inyección 1: Método y versión HTTP inválidos

- **Descripción de la técnica:** Se construyó una conexión TCP hacia el puerto 80 de `servidor_http` mediante Scapy (`SYN → SYN-ACK → ACK`), y sobre dicha conexión se envió una línea de petición que sustituye tanto el método HTTP como la versión de protocolo por cadenas no reconocidas por la especificación RFC 9110.
- **Payload inyectado:** `GEEET / HTTP/9.9\r\nHost: servidor_http\r\n\r\n`
- **Comportamiento esperado:** Se esperaba que Nginx rechazara la solicitud devolviendo un código `400 Bad Request`, o bien cerrara la conexión sin procesar la petición, dado que no reconoce ni el verbo ni la versión de protocolo declarados.
- **Resultado observado:** El servidor no respondió con ningún mensaje HTTP de error. En su lugar, respondió con un segmento TCP con la bandera `RST` activa (`flags = R`), terminando la conexión de forma abrupta a nivel de transporte, sin llegar siquiera a construir una respuesta a nivel de aplicación.
- **Análisis / Hipótesis alternativa:** Nginx, al detectar una malformación grave en la línea de petición (método y versión de protocolo simultáneamente inválidos), prioriza cerrar la conexión inmediatamente por razones de seguridad y eficiencia, evitando invertir recursos en el análisis y construcción de una respuesta de error explícita para una fuente que envía tráfico manifiestamente anómalo.

Inyección 2: Cabecera **Host** sobredimensionada

- **Descripción de la técnica:** Sobre una conexión TCP construida de la misma forma que en la Inyección 1, se envió una petición sintácticamente válida (`GET / HTTP/1.1`), pero con una cabecera `Host` de 9000 caracteres, excediendo ampliamente el tamaño por defecto del buffer de cabeceras de Nginx (directiva `large_client_header_buffers`).
- **Payload inyectado:** `GET / HTTP/1.1\r\nHost: AAAA ... (9000 caracteres) ... \r\n\r\n`
- **Comportamiento esperado:** Se esperaba que Nginx respondiera con `400 Bad Request` o `431 Request Header Fields Too Large`, o cerrara la conexión al exceder el buffer configurado para cabeceras.
- **Resultado observado:** Igual que en la Inyección 1, el servidor respondió únicamente con un segmento TCP `RST`, sin generar una respuesta HTTP de error.
- **Análisis / Hipótesis alternativa:** El resultado refuerza la hipótesis planteada en la Inyección 1: ante entradas que exceden groseramente los límites o el formato esperado por el protocolo, Nginx no distingue entre distintos tipos de anomalía a nivel de aplicación, sino que corta la conexión a nivel de transporte tan pronto detecta que el flujo entrante no puede ser interpretado como una petición HTTP válida dentro de los parámetros configurados.

Modificación de Campos Específicos del Protocolo HTTP

Se realizaron tres modificaciones sobre campos específicos del protocolo HTTP, interceptando el tráfico entre `cliente_http` y `servidor_http` mediante la combinación de ARP spoofing y `NFQUEUE` descrita anteriormente, y reescribiendo el contenido del paquete (recalculando longitudes y *checksums*) antes de reinyectarlo en la red.

Modificación 1: Línea de Estado de la Respuesta

- **Campo modificado:** Línea de estado de la respuesta HTTP (*Status-Line*).
- **Valor original:** `HTTP/1.1 200 OK`.
- **Valor modificado:** `HTTP/1.1 403 Forbidden`.
- **Comportamiento esperado:** Dado que Nginx ya registra la transacción como exitosa antes de que el paquete sea interceptado y modificado, se espera que el servidor no vea afectado su estado interno. En el cliente, en cambio, Wget debería interpretar el nuevo código de error, interrumpir el almacenamiento del recurso e informar el error en consola.

- **Resultado observado:** El comportamiento esperado se confirmó de forma exacta. La consola de Wget mostró: `HTTP request sent, awaiting response... 403 Forbidden` seguido de `ERROR 403: Forbidden`, abortando la descarga. El servidor, por su parte, procesó y respondió la solicitud con normalidad (`200 OK`), sin registrar ninguna anomalía en su propio flujo de ejecución.
- **Análisis:** Este resultado evidencia que la modificación en tránsito fue completamente efectiva e indetectable para ambos extremos de la comunicación, confirmando la vulnerabilidad de HTTP —al no contar con cifrado ni verificación de integridad— frente a ataques de tipo *Man-in-the-Middle*.

Modificación 2: Cabecera **Content-Length**

- **Campo modificado:** Cabecera **Content-Length** de la respuesta.
- **Valor original:** 615 bytes (tamaño real de la página de bienvenida por defecto de Nginx).
- **Valor modificado:** 61 bytes (aproximadamente el 10 % del valor real).
- **Comportamiento esperado:** Se esperaba que Wget leyera únicamente la cantidad de bytes indicada en el header manipulado, descartando el resto del flujo de datos e interpretando que la transacción finalizó correctamente, resultando en una descarga truncada sin mensaje de error visible.
- **Resultado observado:** El comportamiento esperado se confirmó exactamente. Wget reportó `Length: 61 [text/html]` y finalizó indicando `'/dev/null' saved [61/61]`, es decir, dio por completa una descarga que en realidad estaba truncada al 10 % de su tamaño real, sin emitir ningún mensaje de error.
- **Análisis:** Se evidencia un riesgo de integridad silenciosa: Wget confía completamente en el valor declarado por el servidor en la cabecera **Content-Length**, sin verificar de forma independiente la integridad del contenido recibido, por lo que un atacante en la ruta puede trunca contenido sin que el usuario final lo detecte a simple vista.

Modificación 3: Cabecera **Host** de la Solicitud

- **Campo modificado:** Cabecera **Host** de la solicitud del cliente.
- **Valor original:** `Host: servidor_http`.
- **Valor modificado:** `Host: host-inexistente.local`.
- **Comportamiento esperado:** Al no configurarse *virtual hosting* adicional en la imagen de Nginx utilizada, se esperaba que el servidor procesara la solicitud contra su bloque **server** por defecto y respondiera igualmente `200 OK`, evidenciando que el header **Host** no se emplea para enrutamiento en esta configuración particular.
- **Resultado observado:** El comportamiento esperado **no se observó**. La conexión no llegó a completarse: la consola de Wget quedó indefinidamente mostrando `HTTP request sent, awaiting response...` sin recibir jamás una respuesta, mientras que en el contenedor `scapy_mitm` se registraron diez modificaciones consecutivas (`[MODIFICADO #1]` a `[MODIFICADO #10]`) sobre lo que debía ser una única solicitud.
- **Hipótesis alternativa:** A diferencia de las Modificaciones 1 y 2 (que alteran la *respuesta* final del servidor, ya completa y sin más paquetes involucrados en esa transacción), esta modificación altera la *solicitud* del cliente en tránsito, cambiando el largo del payload sin que las pilas TCP de cliente y servidor sincronicen dicho cambio en su propio seguimiento de números de secuencia y confirmación. Esta discordancia se interpretaría como una posible pérdida de segmento, llevando al cliente a reintentar la misma solicitud repetidamente (los diez reintentos observados) sin obtener nunca una respuesta válida. Este resultado ilustra una limitación concreta del método de interceptación basado en NFQUEUE cuando se modifican paquetes intermedios de una transacción aún en curso, en contraste con la modificación de una respuesta final ya completa.

Métricas de Red y Cotas de Desempeño

Se seleccionaron dos métricas de red, distintas a throughput y goodput, que permiten evaluar dos aspectos distintos del enlace: la **latencia (delay)**, asociada al tiempo de propagación y procesamiento de los paquetes, y la **pérdida de paquetes (packet loss)**, asociada a la confiabilidad del canal de comunicación. Ambas métricas se degradaron artificialmente mediante la disciplina de colas **netem** de **tc**, aplicada sobre la interfaz de red del contenedor **cliente_http**. Para cada nivel de degradación se realizaron 5 descargas HTTP consecutivas mediante Wget, registrando el throughput promedio obtenido (en KB/s) y la cantidad de descargas fallidas por *timeout*.

Métrica 1: Latencia (Delay)

- **Definición:** Tiempo que demora un paquete en viajar desde el cliente **cliente_http** hasta el servidor **servidor_http** (o viceversa) a través de la red virtual.
- **Método de modificación:** Se aplicó `tc qdisc add dev eth0 root netem delay {valor}ms` sobre la interfaz del contenedor **cliente_http**.
- **Procedimiento:** Se probaron los siguientes valores de latencia agregada: 0, 25, 50, 100, 150, 200, 300, 500, 800, 1200, 2000, 3000 y 5000 ms.
- **Cota de desempeño:** El servicio mantuvo funcionalidad completa (sin descargas fallidas) hasta los 3000 ms de latencia añadida, aunque con una degradación muy pronunciada del throughput (de 55.26 KB/s en 0 ms a 0.09 KB/s en 3000 ms). A partir de los **5000 ms**, el 100 % de las descargas (5 de 5) fallaron por *timeout*, alcanzando un throughput de 0 KB/s. Se establece la cota de desempeño entre los 3000 ms y los 5000 ms de latencia unidireccional.
- **Gráfico Latencia vs Throughput:** Se presenta en la Figura 1.

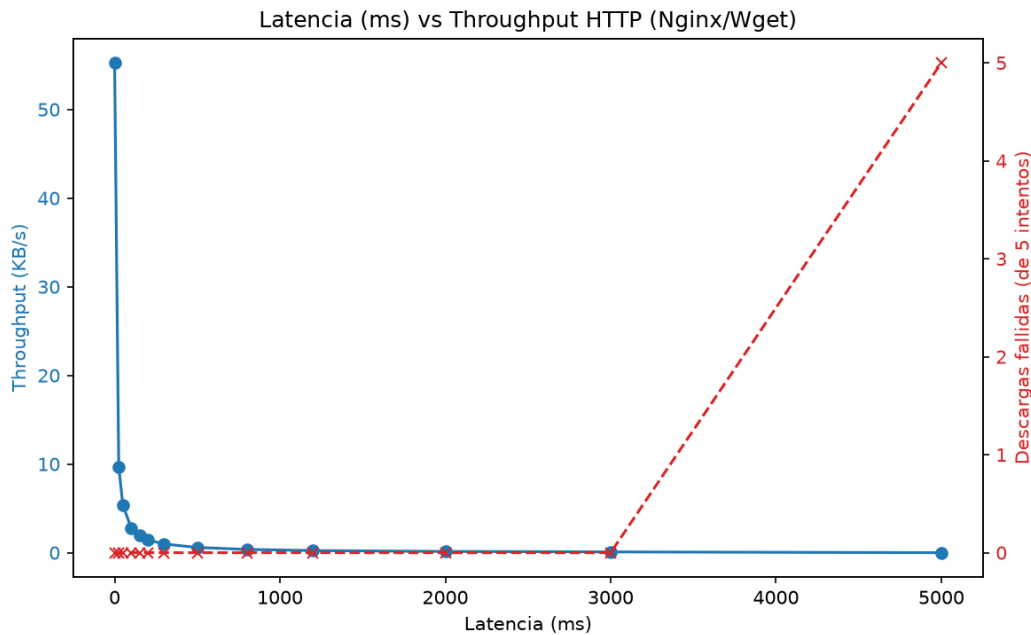


Figura 1: Latencia vs Throughput del enlace HTTP. Se observa una caída sostenida del throughput a medida que aumenta la latencia agregada, hasta la falla total del servicio a partir de los 5000 ms.

- **Análisis:** La relación entre latencia y throughput es marcadamente inversa y no lineal: pequeños incrementos de latencia (0 a 100 ms) ya reducen el throughput en más de un 90 %, lo que se explica por la naturaleza síncrona de HTTP/1.1 sobre una única conexión TCP, donde cada petición debe completar el *three-way handshake* y esperar la respuesta íntegra antes de considerarse finalizada. Recién al superar los 3 segundos de latencia se alcanza el *timeout* configurado en Wget (5 segundos), provocando la falla total del servicio.

Métrica 2: Pérdida de Paquetes (Packet Loss)

- **Definición:** Porcentaje de paquetes que no logran llegar a destino durante la transacción HTTP entre cliente y servidor.
- **Método de modificación:** Se aplicó `tc qdisc add dev eth0 root netem loss {valor}%` sobre la interfaz del contenedor `cliente_http`.
- **Procedimiento:** Se probaron los siguientes porcentajes de pérdida: 0 %, 5 %, 10 %, 20 %, 30 %, 40 %, 50 %, 60 % y 80 %.
- **Cota de desempeño:** El throughput se degrada notoriamente a partir de un 20 % de pérdida de paquetes (caída de 49.46 KB/s en 10 % a 11.21 KB/s en 20 %), y el servicio comienza a presentar fallos completos de conexión a partir de un **60 %** de pérdida (1 de 5 intentos fallidos), empeorando a 2 de 5 fallos con 80 % de pérdida. Se establece la cota de desempeño operativo en torno al 20 % de pérdida, y la cota de falla total en torno al 60 %.
- **Gráfico Pérdida de Paquetes vs Throughput:** Se presenta en la Figura 2.

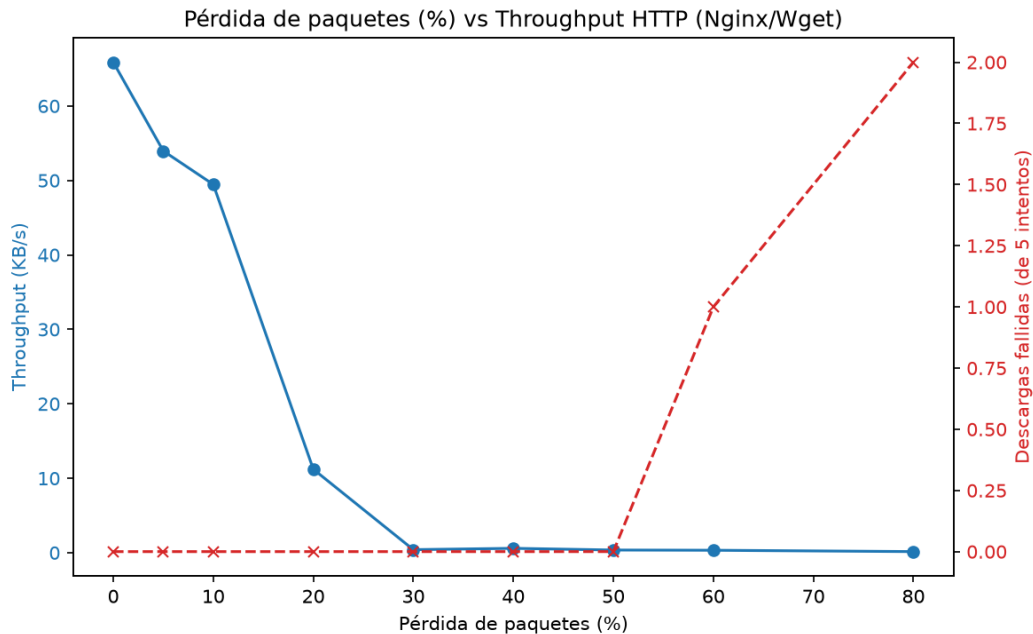


Figura 2: Pérdida de paquetes vs Throughput del enlace HTTP. Se observa una caída abrupta del throughput entre el 10 % y el 20 % de pérdida, y los primeros fallos de conexión a partir del 60 %.

- **Análisis:** A diferencia de la latencia, cuyo efecto sobre el throughput es progresivo desde el primer incremento, la pérdida de paquetes muestra una relativa tolerancia hasta el 10 % (el mecanismo de retransmisión de TCP logra compensar parcialmente los segmentos perdidos), seguida de una caída abrupta al 20 %, punto en el cual la probabilidad acumulada de pérdida durante el *three-way handshake* y el intercambio de datos comienza a impactar fuertemente el tiempo total de la transacción. Los primeros fallos completos (60 % y 80 %) se explican porque, a esos niveles, la probabilidad de que se pierdan simultáneamente varios segmentos críticos del handshake o la retransmisión supera la capacidad de recuperación de TCP dentro del tiempo de espera configurado en Wget.

Conclusiones

El desarrollo de esta segunda etapa permitió pasar de un análisis puramente pasivo del protocolo HTTP, como el realizado en la Tarea 2, a una intervención activa sobre el tráfico mediante Scapy, evidenciando de forma práctica las hipótesis planteadas teóricamente en la sección anterior.

En cuanto a la resiliencia de Nginx frente a tráfico malformado, ambas inyecciones de *fuzzing* (método/versión inválidos y cabecera **Host** sobredimensionada) arrojaron el mismo resultado: el servidor prioriza el cierre abrupto de la conexión a nivel de transporte (**RST**) por sobre la construcción de una respuesta HTTP de error, lo que sugiere una estrategia de descarte temprano de tráfico anómalo antes de invertir recursos de la capa de aplicación en su procesamiento.

Respecto a las modificaciones de campos específicos, dos de las tres hipótesis planteadas en la Tarea 2 se confirmaron exactamente: la alteración de la línea de estado (**200 OK** → **403 Forbidden**) provocó que Wget abortara la descarga reportando el error, mientras que la manipulación de **Content-Length** produjo una descarga silenciosamente truncada sin ningún mensaje de error visible para el usuario. La tercera modificación, sobre la cabecera **Host** de la solicitud del cliente, no siguió el comportamiento esperado: en lugar de que Nginx respondiera igual contra su *vhost* por defecto, la conexión entró en un ciclo de reintentos sin respuesta, evidenciando una limitación del método de intercepción al modificar paquetes intermedios de una transacción TCP aún en curso, en contraste con la modificación exitosa de respuestas ya completas.

Finalmente, la evaluación de las dos métricas de red seleccionadas —latencia y pérdida de paquetes— permitió identificar cotas de desempeño concretas para el servicio HTTP estudiado: una falla total a partir de los 5000 ms de latencia agregada, y a partir de un 60 % de pérdida de paquetes, con una degradación progresiva y no lineal del throughput en ambos casos desde niveles muy inferiores a dichas cotas. En conjunto, los resultados de esta tarea refuerzan la importancia de los campos de control de HTTP y de las condiciones subyacentes de la capa de transporte y de red para garantizar la correcta e íntegra entrega de contenido web, así como la relevancia de mecanismos de cifrado e integridad (como HTTPS/TLS) que HTTP, en su forma no cifrada, no provee.

Referencias

1. Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP Semantics (RFC 9110)*. Internet Engineering Task Force (IETF).
2. Docker Documentation. (2026). *Docker Compose CLI reference and network architecture options*. Recuperado de <https://docs.docker.com/>
3. Nginx Reference Documentation. (2026). *Nginx HTTP Core Module Architecture*. Recuperado de <https://nginx.org/en/docs/>
4. Free Software Foundation. (2025). *GNU Wget Manual*. Recuperado de <https://www.gnu.org/software/wget/manual/>
5. Wireshark Foundation. (2026). *Wireshark User Guide*. Recuperado de <https://www.wireshark.org/docs/>
6. Scapy Documentation. (2026). *Scapy Usage Guide*. Recuperado de <https://scapy.readthedocs.io/>
7. Netfilter Project. (2026). *libnetfilter_queue and NFQUEUE target documentation*. Recuperado de https://www.netfilter.org/projects/libnetfilter_queue/
8. Linux Foundation. (2026). *tc-netem(8) — Linux manual page*. Recuperado de <https://man7.org/linux/man-pages/man8/tc-netem.8.html>

Anexos

Con el fin de preservar el orden formal del documento y dar cumplimiento irrestricto a los criterios metodológicos estipulados por la Facultad de Ingeniería y Ciencias de la Universidad Diego Portales, todas las evidencias visuales, registros gráficos de instalación y capturas de flujos de datos se agrupan de forma exclusiva dentro de esta sección analítica, habiendo sido debidamente referenciados a lo largo del texto de desarrollo principal.

Evidencias de la Tarea 2: Configuración e Infraestructura

```
[pepo@Pepo-Linux tarea2]$ docker build -t ubuntu-nginx ./servidor
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:latest
--> f3d28607ddd7
Step 2/4 : RUN apt-get update && apt-get install -y nginx
--> Using cache
--> 958d76b21a7e
Step 3/4 : EXPOSE 80
--> Using cache
--> 69d9d05e8450
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
--> Using cache
--> a8dda080b7bc
Successfully built a8dda080b7bc
Successfully tagged ubuntu-nginx:latest
[pepo@Pepo-Linux tarea2]$ docker build -t ubuntu-wget ./cliente
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM ubuntu:latest
--> f3d28607ddd7
Step 2/3 : RUN apt-get update && apt-get install -y wget
--> Using cache
--> 157a13e9ecfd
Step 3/3 : CMD ["tail", "-f", "/dev/null"]
--> Using cache
--> 2993ec30578a
Successfully built 2993ec30578a
Successfully tagged ubuntu-wget:latest
[pepo@Pepo-Linux tarea2]$
```

Figura 3: Captura del proceso de construcción de imágenes Docker a partir de los Dockerfile creados.

```
[pepo@Pepo-Linux tarea2]$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
81ce1b9b5e79   ubuntu-wget    "tail -f /dev/null"     27 minutes ago Up 27 minutes      cliente_http
a835a6a39375   ubuntu-nginx   "nginx -g 'daemon of..." 28 minutes ago Up 28 minutes      80/tcp        servidor_http
```

Figura 4: Ejecución de contenedores y prueba de conexión desde el cliente Wget en la terminal.

Evidencias de la Tarea 2: Captura Forense de Paquetes en Wireshark

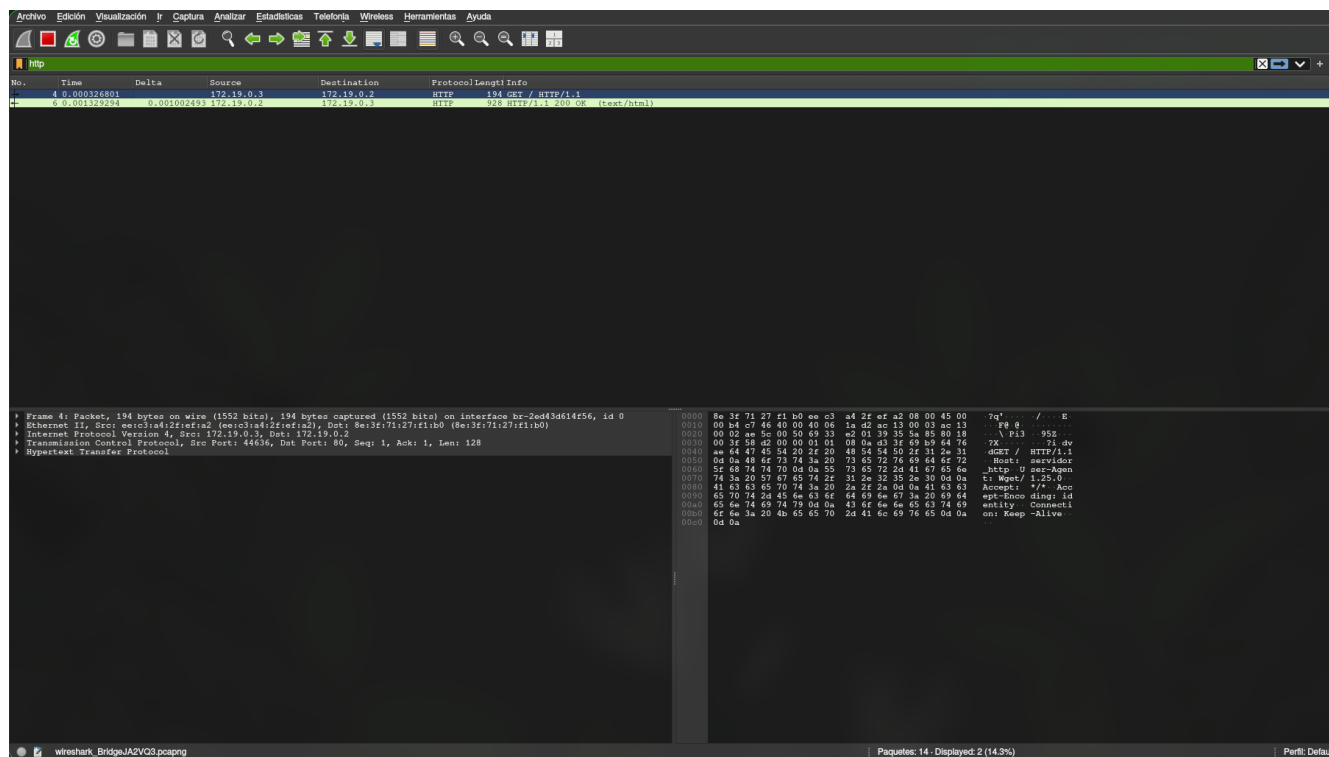


Figura 5: Captura del tráfico HTTP en Wireshark evidenciando los paquetes GET y 200 OK.

Evidencias de la Tarea 3: Scapy, Inyecciones y Modificaciones

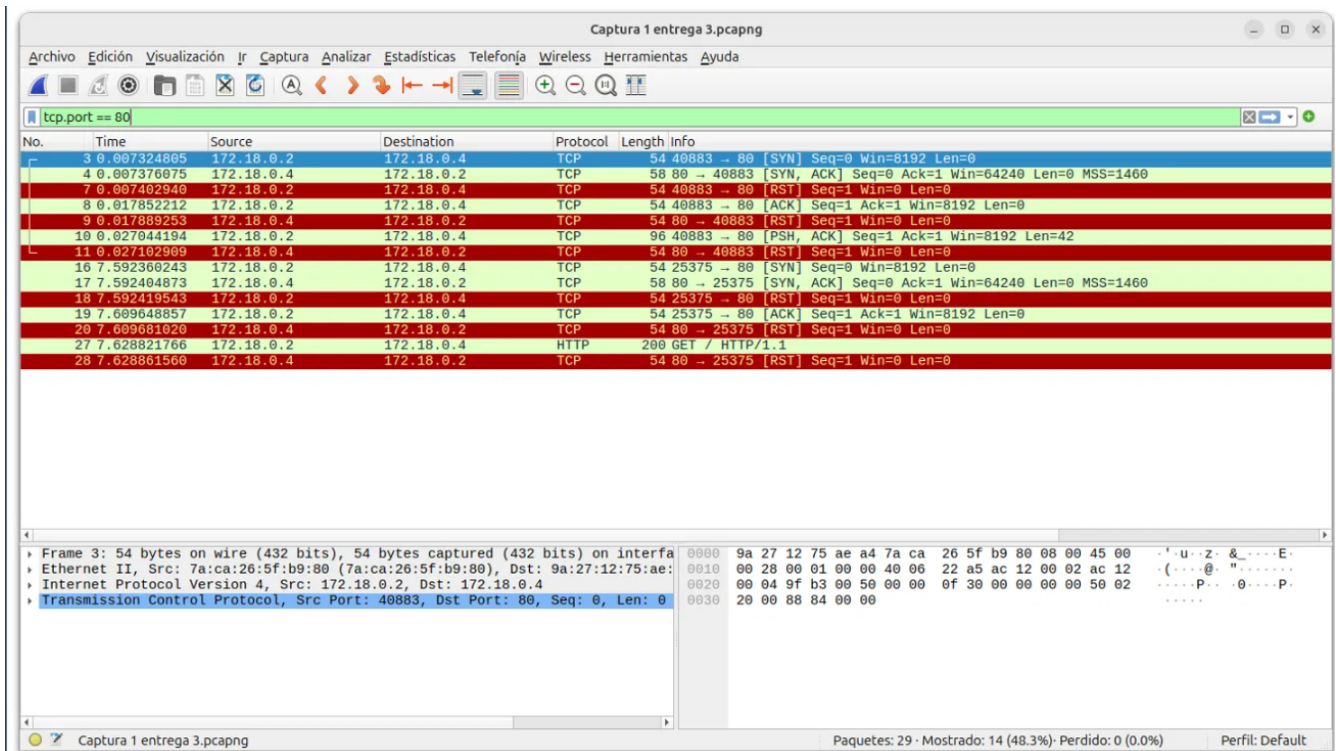


Figura 6: Captura de Wireshark durante las dos inyecciones de fuzzing, mostrando los segmentos TCP RST devueltos por el servidor.

```
[C-COMANDOS] $ docker exec cliente_http wget -O /dev/null http://servidor_http
--2026-07-04 01:29:33-- http://servidor_http/
Resolving servidor_http (servidor_http)... 172.18.0.4
Connecting to servidor_http (servidor_http)|172.18.0.4|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 61 [text/html]
Saving to: '/dev/null'
```

Figura 7: Consola de Wget mostrando la descarga truncada a 61 bytes tras la modificación de Content-Length.

Video Público de la Tarea 3

<https://youtu.be/Hv-c5St5hl4>

Github de la tarea

<https://github.com/Nico-gzz/tarea-2-taller-de-redes/tree/main>